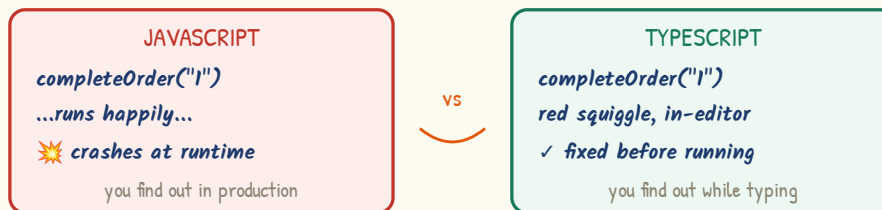


FULL STACK DEVELOPER PATH

Chapter 6

TypeScript

JavaScript · plus types · errors before you run



SOURCE : freeCodeCamp × Scrimba – “Become a Fullstack Developer from Scratch”

MODULES : TypeScript fundamentals (40:12:26) · TypeScript in React (42:15:21)

COVERS : annotations · custom types · unions · narrowing · utility types · typing components

PROJECTS : Pizza restaurant · Assembly Endgame (React)

⚡ THE KEY IDEA

TypeScript **doesn't make your job harder** – it significantly improves your ability to add features and debug. Change one type and TypeScript **immediately shows you every place** that change breaks.

1 Telling TypeScript what you mean ★★★★★

// primitives — annotate with a colon

let name: string = "Bob";

let age: number = 42;

let isStudent: boolean = false;

// function parameters — where it really pays off

function completeOrder(orderId: number) { ... }

completeOrder("1"); // X string "1" is not assignable to a number

⚡ A FEW CHARACTERS BUY A LOT

Adding **: number** to one parameter is enough for TypeScript to catch every wrong call site in the program. **We** are no longer crashing with type errors — the mistake is reported where it is made, not where it explodes.

► Inference — and the let / const twist

let myName = "Bob";

// hover → let myName: string

const myName2 = "Bob";

// hover → const myName2: "Bob"

TypeScript already **infers** a type when you assign a value — you don't have to annotate everything. But **let** and **const** infer differently, and the reason is logical.

DEFINITION — LITERAL TYPE

Not a generic **string** (which would allow any string) but an **actual value type** — the string **"Bob"** specifically. A **let** can be reassigned, so TypeScript restricts it to **the type**. A **const** can't be reassigned, so TypeScript pins it to **the value**.

HOVER IS THE LEARNING TOOL

The IntelliSense pop-up tells you exactly what TypeScript has inferred. Hovering over a variable, a parameter, or even the **=>** of an arrow function is **how you check your understanding** — and how you discover the type you were about to write by hand.

⚠ TYPESCRIPT DOES NOT STOP YOUR CODE RUNNING

A red squiggle is a **warning**, not a wall. You can save, run, and get output with errors still on screen — the checking happens **at compile time, alongside you**, not at runtime. It can be configured to block a build, but by default nothing forces you to listen.

2 The type keyword

★★★★★

DEFINITION

The **type** keyword exists **only in TypeScript**. By convention the name starts with a **capital letter**. Aliasing a primitive (**type Food = string**) is legal but not very useful – the payoff comes with **object types**.

```
type Person = {
  name: string
  age: number
  isStudent: boolean
}
```

```
const person1: Person = { name: "Jack", age: 44, isStudent: true };
const person2: Person = { name: "Jill", age: 66, isStudent: false };
//                                     ↑ X caught: capital I
```

⚡ WHAT AN OBJECT TYPE ACTUALLY BUYS

The worked example has two near-identical objects, one written **isStudent** and the other **IsStudent**. As plain JavaScript **there is no error** – but it is “pretty ripe for a bug down the road”. Defining the shape once **forces you to stay consistent** and surfaces the typo immediately.

SEPARATORS ARE FREE

Inside an object type you may separate properties with **commas**, **semicolons**, or **nothing at all**. Pick one and stay consistent within the project.

⚠ IT COSTS LINES

Writing the type is extra work and “can feel like it’s breaking your flow”. The trade is **five lines now** against a silent bug later.

► Arrays

```
// the syntax we use
const people: Person[] = [];
const menu: Pizza[] = [];
```

```
// equivalent, generic syntax
const people: Array<Person> = [];
```

⚠ implicit any

An un-annotated function parameter makes TypeScript warn that it **implicitly has an any type** – meaning all checking is switched off for it. Annotating the parameter (**pizzaObj: Pizza**) is what turns the protection back on.

⚡ FIND THE RED SQUIGGLES, THEN FIX THEM

The working method throughout the course is one loop: **add one type, save, and see what lights up**. Typing **orderQueue** as **Order[]** resolves a cluster of unrelated-looking warnings at once, because TypeScript can finally reason about every operation performed on it. **One annotation often fixes several complaints**.

3 One value, several types

★★★★★

```
function getPizzaDetail(identifier: string | number) { ... }
```

The single pipe reads as the word *or*. It works with literal types and with general types alike.

► Type narrowing

```
if (typeof identifier === "string") {
  return menu.find(pizza =>
    pizza.name.toLowerCase() === identifier.toLowerCase()
  );
} else if (typeof identifier === "number") {
  return menu.find(pizza => pizza.id === identifier);
} else {
  throw new TypeError("identifier must be a string or a number");
}
```

DEFINITION — TYPE NARROWING

When you don't know a value's type up front, TypeScript expects you to narrow it down and handle each case. Inside the string branch it knows the value is a string; in the next branch it has *narrowed* it to a number.

⚡ TYPESCRIPT IS READING YOUR CONTROL FLOW

Write `identifier.toLowerCase()` in the number branch and it says "toLowerCase does not exist on type number" — it parsed your `if` and knows the string case is already handled.

Copy a line down and forget to change `name` to `id`, and it warns that you can't `===` compare a string with a number. *Two real bugs caught by the same mechanism.*

WHY else if BEATS A BARE else

A plain `else` would satisfy TypeScript — but your compiled code can be imported by a *plain JavaScript file*, which gets no compile-time protection at all. Someone can call `getPizzaDetail(false)` and silently receive `undefined`. Being explicit — `else if (typeof ... === "number")` plus an `else` that throws — gives that developer *an intentional feedback loop*. Be as explicit as you can, within reason.

⚠ TYPES VANISH AT THE BOUNDARY

This is the wider lesson. TypeScript protects the files it checks — the moment a value crosses into plain JavaScript, *all guarantees are gone*. Defensive runtime checks at the edges of your module are not redundant with the type system; they cover exactly the ground the type system *cannot reach*.

4 possibly undefined

★★★★★

⚠ THE WARNING YOU WILL SEE MOST

Object is possibly undefined. It appears whenever you use the result of `.find()` without checking it. TypeScript knows that if `find` doesn't find anything it returns `undefined` – and you haven't handled that.

```
const order = orderQueue.find(o => o.id === orderId);
// hover → const order: Order | undefined

if (!order) {
  console.error("Order ID was not found in the order queue");
  return;           // ← halts execution, so TypeScript is satisfied
}

order.status = "completed"; // ✓ order is now definitely an Order
```

- `return` early, or
 - `throw new Error(...)` – either works,
 - because both *halt execution*.
- Once the impossible case can't reach the next line, TypeScript narrows away the `undefined` and the warning disappears.

⚠ NO WARNINGS ≠ NO BUGS

An empty problems panel *does not automatically mean your program works as expected*, nor that every edge case is handled. What it means is that for *relatively little effort* you have a much greater degree of confidence.

► The refactor demo – why any of this is worth it

⚡ CHANGE THE TYPE, FOLLOW THE ERRORS

Add `id: number` to the `Pizza` type and every existing call that omits an `id` *lights up red immediately*. Rename `orderQueue` to `orderHistory` and TypeScript lists exactly what must change.

That is the whole argument: *TypeScript turns a risky refactor into a checklist*. The fix "will not always be quite this easy" – but you are never guessing where the damage is.

GET YOUR HANDS ON IT

The instructor deliberately leaves a rename half-finished so the reader has to fix the resulting errors. That is the intended way to learn this: make a *small, arbitrary change* to a type in a working project and follow the trail. You learn more from *watching TypeScript object* than from reading about it.

5 Deriving one type from another ★★★★★

A recurring problem: `addNewPizza` should receive a pizza *without* an `id` – the function generates that itself – but the pizza it pushes must be complete.

Utility	What it produces
<code>Partial<T></code>	Every property becomes <i>optional</i> . Too loose here – the caller could omit everything
<code>Omit<T, K></code>	A brand-new type with the named properties <i>removed</i> , everything else still required

```
function addNewPizza(pizzaObj: Omit<Pizza, "id">): Pizza {
  const newPizza: Pizza = {
    id: nextPizzaId++,
    ...pizzaObj
  };
  menu.push(newPizza);
  return newPizza;
}
```

// omit several at once with a union of string literals
`Omit<User, "id" | "username">`

WHY OMIT AND NOT PARTIAL

We *want to require* the caller to supply everything except the one property the function owns. With `Partial` you would have to add "a bunch of extra checking inside the body of the function" – a road worth not going down.

⚠️ A COURSE-SPECIFIC QUIRK

`Omit` was introduced in *TypeScript 3.5*. At recording time Scrimba ran an older version, so `Omit` shows a red squiggle *in the course environment only* – the instructor demonstrates the correct behaviour with screenshots from VS Code. Your editor will be fine.

⚡ THERE ARE MANY MORE

The course deliberately does not teach every utility type, "because the documentation is very straightforward". Knowing that *a family of type-transforming helpers exists* – and that `Omit` and `Pick` are the ones you reach for first – is the actual lesson.

THE PRINCIPLE – ONE SOURCE OF TRUTH

Utility types exist so a related type is *derived rather than duplicated*. Hand-writing a second near-identical type means that when the original gains a property, the copy silently falls out of date. `Omit<Pizza, "id">` can never drift, because it *is* `Pizza`.

6 Typing components

★★★★★

```
import type { JSX } from "react";
import type { Language } from "../languages";

// naming convention: ComponentName + Props
type LanguageChipsProps = {
  languages: Language[]
  wrongGuessCount: number
}

export default function LanguageChips(
  { languages, wrongGuessCount }: LanguageChipsProps
): JSX.Element {

  const isLanguageLost: boolean = ...
  const className: string = ...

  const elements: JSX.Element[] = languages.map(
    (lang: Language, index: number): JSX.Element => ...
  );
}
```

Thing being typed	Its type
A component's return	<code>JSX.Element</code> – or <code>JSX.Element null</code> if it can render nothing
Props	One custom type named <code><Component>Props</code>
A <code>.map()</code> callback	Params <code>(item: T, index: number)</code> , returns <code>JSX.Element</code>
The mapped result	<code>JSX.Element[]</code> – map returns an array
A function prop	<code>startNewGame: () => void</code>

SYNTAX – A FUNCTION AS A PROP

Prop name, then **parentheses for the parameters**, then an **arrow**, then the **return type**. A callback that returns nothing is typed `() => void`. This is the one people stall on – the others are just `string`, `number`, `boolean`.

SHARING A TYPE BETWEEN FILES

Nothing exotic – add `export` to the type where it is defined and `import type { Language } from "../languages"` where it is needed. It is a **plain JavaScript export**. Derive prop types from existing ones with `Omit<Language, "name">` rather than re-declaring the same fields.

7 QUICK REVISION

Thing	The one-line answer
Annotation syntax	<code>name: string</code> – a colon, then the type
Inference	TypeScript works most types out for you; annotate what it can't
<code>let</code> vs <code>const</code>	<code>let</code> infers <code>string</code> ; <code>const</code> infers the literal <code>"Bob"</code>
<code>type</code> keyword	TypeScript-only. Capitalised by convention
Object type	Property names paired with types, not values
Separators	Comma, semicolon or nothing – just be consistent
Arrays	<code>Person[]</code> or <code>Array<Person></code> – identical
<code>any</code>	An implicit <code>any</code> warning means checking is off for that value
Union	<code>string number</code> . Read the pipe as or
Narrowing	<code>typeof</code> checks; TypeScript follows your control flow
<code>X undefined</code>	What <code>.find()</code> returns. Handle it or you get a warning
Satisfying it	<code>return</code> early or <code>throw</code> – both halt execution
<code>Partial<T></code>	All properties optional – usually too loose
<code>Omit<T, "k"></code>	T without key k. Union the keys to drop several
Component return	<code>JSX.Element</code> , or <code>JSX.Element null</code>
Props type	Named <code><Component>Props</code> by convention
Function prop	<code>onClick: () => void</code>
Sharing types	<code>export</code> the type, <code>import type</code> it

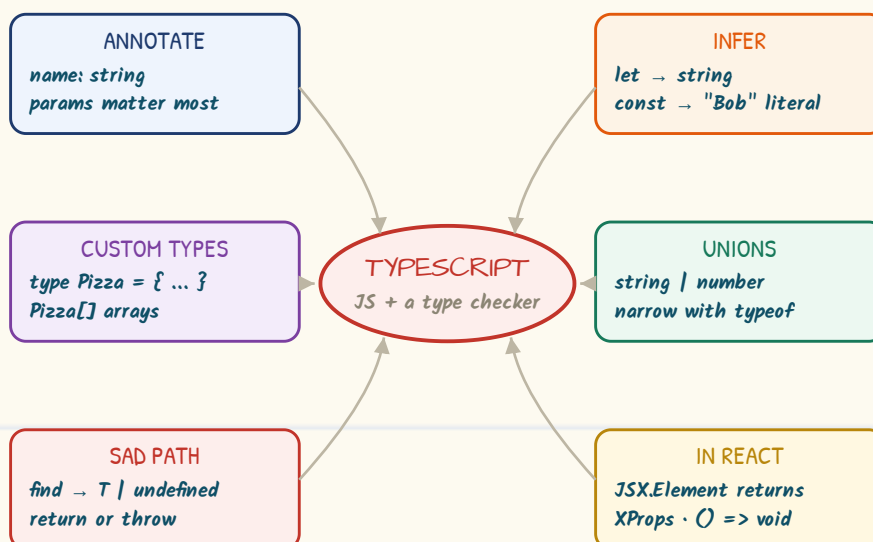
⚡ THE FOUR MOST FORGETTABLE DETAILS

1. `const` infers a literal type, not a general one. 2. A bare `else` leaves plain-JavaScript callers unprotected – prefer an explicit `else if` plus a throwing `else`. 3. `.find()` can always return `undefined`. 4. Zero warnings is not proof of correctness.

THE HABIT TO BUILD

Hover over things. The IntelliSense pop-up is the fastest way to learn what TypeScript already knows – and it is how you find the type you were about to guess.

8 MIND MAP



⚡ THE THROUGH-LINE

Every feature here answers the same question: what shape is this value, and have you handled every shape it could be? Annotations state the shape, custom types name it, unions admit several, narrowing proves which one you have, and utility types derive one shape from another.

Syntax	Means
<code>x: string</code>	Annotation – x is a string
<code>string number</code>	Union – either one. Read the pipe as or
<code>Person[]</code>	An array of Person
<code>Order undefined</code>	What a failed <code>.find()</code> gives you
<code>Omit<T, "k"></code>	T, minus key k
<code>() => void</code>	A function taking nothing, returning nothing
<code>JSX.Element</code>	What a React component returns
<code>import type</code>	Bring a type across files – a plain export underneath

Next up → *Next.js*, where these types meet a server.